Lab S09 · Que el aviso no tumbe la transferencia

Mensajería con RabbitMQ, consumidor idempotente, DLQ y reto Outbox. Partí de la rama 05-mensajeria-base de Mexi Banco. Ejecuté las pruebas con Docker Compose y capturé la terminal usada para consultar respuestas, colas, logs y bases de datos.

1. El problema inicial

Apagué notificacion y transferí 50. El cliente recibió HTTP 500, pero la saga quedó en ABONADA: Ana bajó de 1000 a 950 y Beto subió de 500 a 550. El dinero ya se había movido; falló la llamada HTTP síncrona que esperaba la saga para enviar los avisos.



Figura 1. Captura de pantalla: error, estado ABONADA y saldos de ambas cuentas.

2. Publicar un hecho, no llamar por HTTP

La transferencia genera un eventoId estable y guarda el estado final junto con el evento en la tabla outbox_evento (RegistroDeEventos). ReleDeEventos lee esa tabla y publica un JSON en el exchange durable transferencias con routing key transferencia.realizada o transferencia.revertida. El servicio ya no llama a notificación por HTTP.

Probé el exchange sin cola enlazada: la transferencia respondió 201 y quedó COMPLETADA. Como el mensaje no tenía ruta, mandatory hizo que el publicador detectara el retorno y el Outbox conservó la fila pendiente. Un publisher confirm confirma que el broker aceptó la publicación, pero no prueba que un consumidor la procesó ni hace atómica la escritura entre base y broker.



Figura 2. Captura de pantalla: exchange topic durable, 201 y evento pendiente sin cola.

3. Desacople en tiempo

Notificación declara una cola durable enlazada con transferencia.*. Con el consumidor apagado hice tres transferencias de 11, 12 y 13: todas respondieron 201 y la cola acumuló tres mensajes Ready.

```
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped                                3.8s

--- Tres transferencias con consumidor apagado ---
Monto 11: HTTP 201
Monto 12: HTTP 201
Monto 13: HTTP 201

--- Cola mientras notificacion esta apagada ---
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias.dlq 0       0
notificacion.transferencias     3       0

--- Exchange y tipo ---
transferencias  topic   true
transferencias.dlx      direct  true
```

Figura 3. Captura de pantalla: tres HTTP 201 y tres mensajes Ready.

Al encender notificación, la cola bajó a cero. El consumidor guardó dos avisos por cada evento realizado y envió el ack después del commit.

```
[+] start 3/3
 ✔ Container mexi-banco-v06-rabbitmq-1  Healthy                                   0.5s
 ✔ Container mexi-banco-v06-db-1        Healthy                                   0.5s
 ✔ Container mexi-banco-v06-discovery-1 Healthy                                   0.5s

--- Cola al regresar notificacion ---
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias.dlq 0       0
notificacion.transferencias     0       0

--- Avisos recientes de Beto ---
TRANSFERENCIA_RECIBIDA por 13, saldo resultante 586.00
TRANSFERENCIA_RECIBIDA por 12, saldo resultante 573.00
TRANSFERENCIA_RECIBIDA por 11, saldo resultante 561.00

--- Avisos recientes de Ana ---
TRANSFERENCIA_ENVIADA por 13, saldo resultante 914.00
TRANSFERENCIA_ENVIADA por 12, saldo resultante 927.00
TRANSFERENCIA_ENVIADA por 11, saldo resultante 939.00
```

Figura 4. Captura de pantalla: cola vacía y tres avisos en cada bandeja.

4. La reversión también avisa

Provocé la falla de abono hacia Beto mediante FALLAR_ABONO_A_CLABE. La saga devolvió 150 a Ana y terminó en COMPENSADA. El evento TransferenciaRevertida generó un único aviso de devolución a la cuenta de origen.

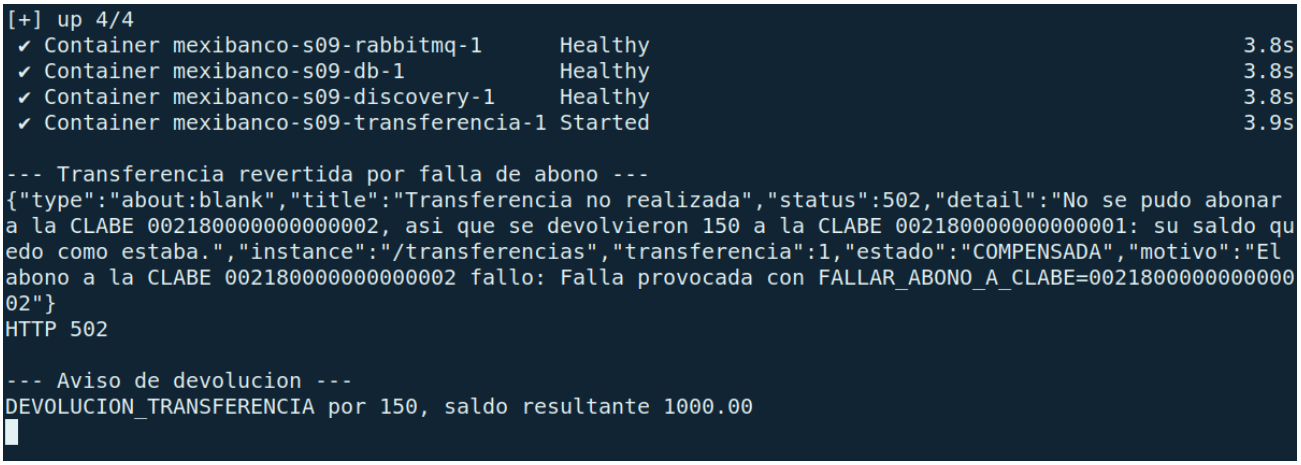


Figura 5. Captura de pantalla: respuesta 502 con COMPENSADA y aviso de devolución.

5. Un mismo mensaje puede llegar dos veces

Republiqué un JSON con el mismo eventoId. Para mostrar el problema, probé temporalmente el consumidor sin la marca de idempotencia: el conteo de avisos para ese monto pasó de 1 a 2. Luego restauré la versión final: la inserción de eventoId con llave primaria y los avisos ocurren en una sola transacción; si el ID ya existe, se crean cero avisos y se confirma el mensaje igualmente.

```
--- Sin proteccion contra duplicados ---
eventoId=32d14846-b0fb-4cdb-a2aa-114c8952e686
Avisos antes: 1

Republicacion: {"routed":true}
Avisos despues: 2
notificacion-1  | 2026-10-23T03:21:05.065Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 2 (evento 32d
14846-b0fb-4cdb-a2aa-114c8952e686, redelivered=false): 2 aviso(s) nuevo(s)
```

Figura 6. Captura de pantalla: sin protección, el segundo mensaje duplicó el aviso.

```
--- Transferencia normal despues del veneno ---
{
  "id": 2,
  "estado": "COMPLETADA",
  "eventoId": "32d14846-b0fb-4cdb-a2aa-114c8952e686"
}

--- Avisos antes de repetir el evento ---
1

--- Republicar el mismo eventoId ---
eventoId=32d14846-b0fb-4cdb-a2aa-114c8952e686
{"routed":true}
--- Avisos despues de la entrega duplicada ---
1
notificacion-1  | 2026-10-23T03:20:16.734Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 2 (evento 32d
14846-b0fb-4cdb-a2aa-114c8952e686, redelivered=false): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-23T03:20:18.967Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 2 (evento 32d
14846-b0fb-4cdb-a2aa-114c8952e686, redelivered=false): 0 aviso(s) nuevo(s)
```

Figura 7. Captura de pantalla: con idempotencia, una repetición produjo cero avisos nuevos.

6. Mensaje envenenado y DLQ

Publiqué un JSON incompleto. Con reencolado temporal, el consumidor falló varias veces con el mismo mensaje y la cola lo mantuvo Unacked o reentregándolo: no podía avanzar limpiamente. Restauré el nack sin requeue. La cola principal usa transferencias.dlx y la clave notificacion.transferencias; RabbitMQ apartó el mensaje en notificacion.transferencias.dlq con x-death: rejected. Después hice otra transferencia normal de 5: respondió 201 y produjo su aviso.

```
--- Antes del rechazo sin requeue ---
{"routed":true}
--- El mismo mensaje falla repetidamente ---
notificacion-1  | 2026-10-23T03:23:56.124Z  WARN 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] Evento invalido; se reencola: Unexpected end-of-input wi
thin/between Object entries
notificacion-1  | 2026-10-23T03:23:56.625Z  WARN 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] Evento invalido; se reencola: Unexpected end-of-input wi
thin/between Object entries
notificacion-1  | 2026-10-23T03:23:57.130Z  WARN 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] Evento invalido; se reencola: Unexpected end-of-input wi
thin/between Object entries
notificacion-1  | 2026-10-23T03:23:57.632Z  WARN 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] Evento invalido; se reencola: Unexpected end-of-input wi
thin/between Object entries

--- Cola principal ---
notificacion.transferencias.dlq 0       0
notificacion.transferencias     0       1
```

Figura 8. Captura de pantalla: el JSON inválido se reencolaba repetidamente.

```
--- Con nack sin reencolar ---
notificacion-1  | 2026-10-23T03:24:25.757Z  WARN 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] Evento invalido; enviado a DLQ: Unexpected end-of-input
within/between Object entries

--- El broker aparto el mensaje ---
notificacion.transferencias.dlq 1       0
notificacion.transferencias     0       0

--- Cabecera x-death ---
rejected
```

Figura 9. Captura de pantalla: mensaje rechazado en la DLQ y cola principal libre.

7. Reto extra: Transactional Outbox

Apagué RabbitMQ y transferí 1. La respuesta fue 201, el estado quedó COMPLETADA y la fila de Outbox permaneció pendiente. Al iniciar RabbitMQ, el relay la publicó por sí solo y notificación guardó el aviso. Guardar estado y evento en una transacción local elimina la ventana entre esas dos escrituras. Si el relay publica y cae antes de marcar la fila, puede reenviar el mismo evento; por eso el consumidor usa eventoId.

```
[+] stop 1/1
 ✔ Container mexibanco-s09-rabbitmq-1 Stopped                                1.3s

--- Broker apagado: transferencia de 1 ---
{"id":3,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"20
26-10-23T03:21:51.003964736Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":992.00,"int
entosDeCompensacion":0,"actualizada":"2026-10-23T03:21:51.073447270Z","eventoId":"cd9a1be7-eed6-43b1
-9ce9-66c09e5a368c"}
HTTP 201

--- Filas de Outbox pendientes ---
cd9a1be7-eed6-43b1-9ce9-66c09e5a368c|t
```

Figura 10. Captura de pantalla: RabbitMQ apagado, 201 y evento pendiente.

```
--- Outbox tras recuperar RabbitMQ ---
cd9a1be7-eed6-43b1-9ce9-66c09e5a368c|t

--- Aviso que llego sin republicar a mano ---
TRANSFERENCIA_RECIBIDA por 1, saldo resultante 508.00
```

Figura 11. Captura de pantalla: fila publicada y aviso recibido al volver RabbitMQ.

8. Preguntas de cierre

1. Si RabbitMQ está caído, ¿qué pasa con el aviso? En una versión que guarda COMPLETADA y después publica directamente, se pierde: el confirm puede detectar que el broker no confirmó, pero no vuelve a ejecutar una publicación que nunca salió tras una caída del proceso. En mi versión final, el Outbox conserva el evento y el relay lo manda al recuperarse el broker.

2. ¿Por qué el ack va después de guardar? Si el consumidor cae antes del ack, RabbitMQ vuelve a entregar el mensaje: puede repetirse, pero no se pierde, y eventoId evita duplicar avisos. Si hiciera ack primero y cayera antes del commit, el broker lo borraría y el aviso faltaría para siempre.

3. Elección y sacrificio. Elegí que mover dinero no dependa de que el servicio de avisos esté disponible. Lo hice porque un fallo del consumidor no debe cambiar el resultado de la saga. Sacrifico la confirmación inmediata de que el cliente ya recibió el aviso: ahora llega eventualmente y hay que vigilar cola, DLQ y Outbox.

Verificación: 22 pruebas de transferencia y 8 de notificación pasaron. Las pruebas de integración usaron Docker Compose, curl, RabbitMQ y PostgreSQL. Para evitar modificar datos de laboratorios previos, repetí las pruebas finales en el proyecto Docker aislado mexibanco-s09.

Fuente: consigna local del Lab S09 (README y guía HTML) y código de la rama 05-mensajeria-base del repositorio Mexi Banco. Las observaciones y capturas corresponden a esta ejecución.